

Research Excellence Framework Optimisation

Problem

Bryn Elliott

Supervisor: Dr Nir Oren

CS4529: Single Honours Computing Project

✉ b.elliott.21@abdn.ac.uk



What is REF?

- REF is the Research Excellence Framework.
- it is a review of research from Higher Education Institutions (HEIs) carried out by the UKRI (UK Research and Innovation) on behalf of the 4 major funding bodies in the UK.
- The outcome of the review dictates funding for HEIs for the subsequent years.



What's the Problem?

- The REF places limits on the amount of research an HEI can submit.
- This, along with the correlation between average research score and funding, create an opportunity for optimisation.

What's the Current Solution and Why Doesn't it Suffice?

- Traditionally, research would be chosen by internal scores assigned to papers, mirroring a greedy algorithm.
- This, while generating a reasonably high-scoring submission, has a major limitation.
- Greedy Algorithms have a tendency to fall into what are called local maximums.
- These occur when optimisation is carried out linearly, reaching a point where, while there are better solutions, every solution that is one step away from the current is worse than the current.
- Greedy algorithms take this as a global maximum, the best possible solution

How Can it be Solved?

- If instead, a genetic algorithm is used, these local maximums can be avoided and the global maximum can be more consistently reached.
- Figure 1 illustrates this:

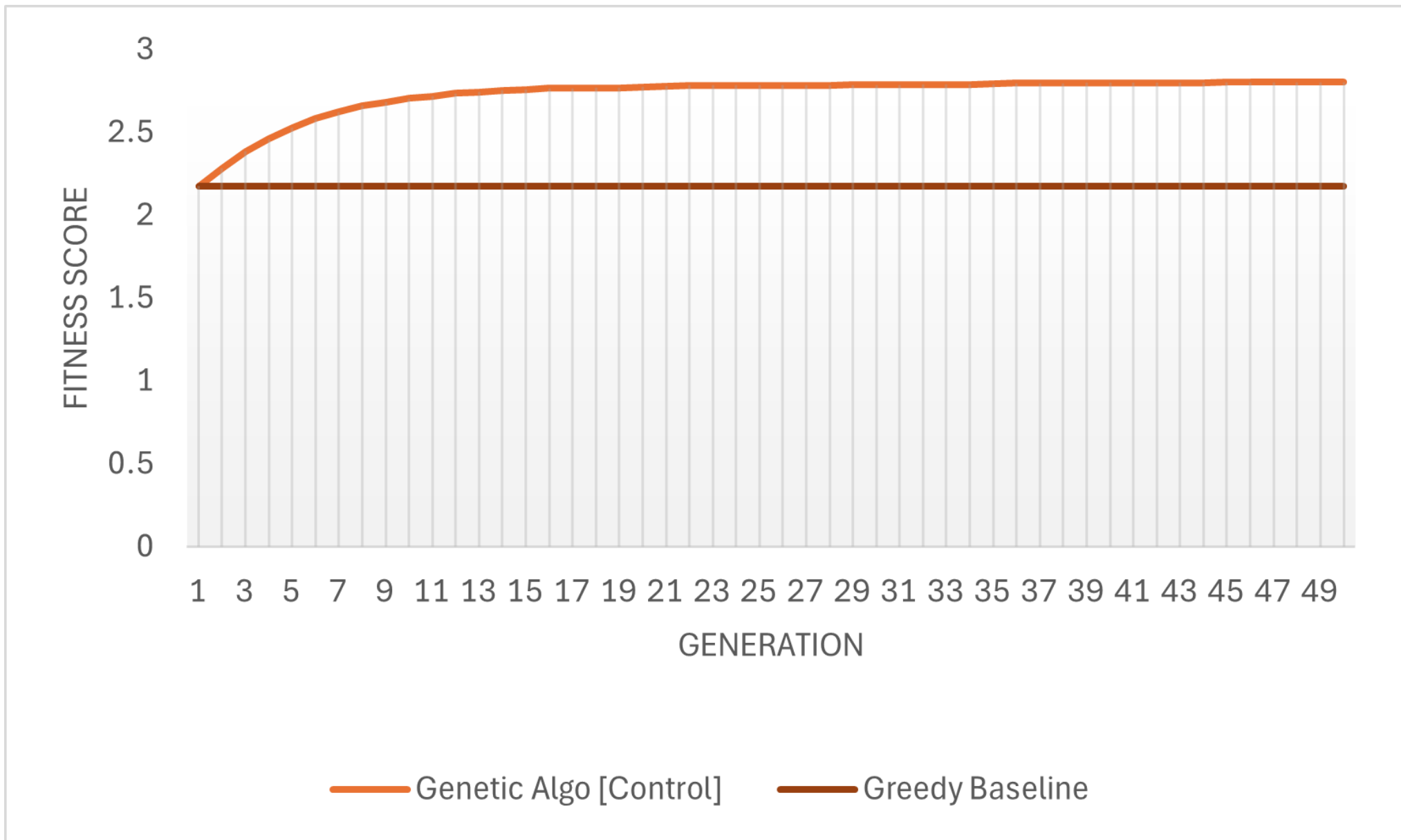


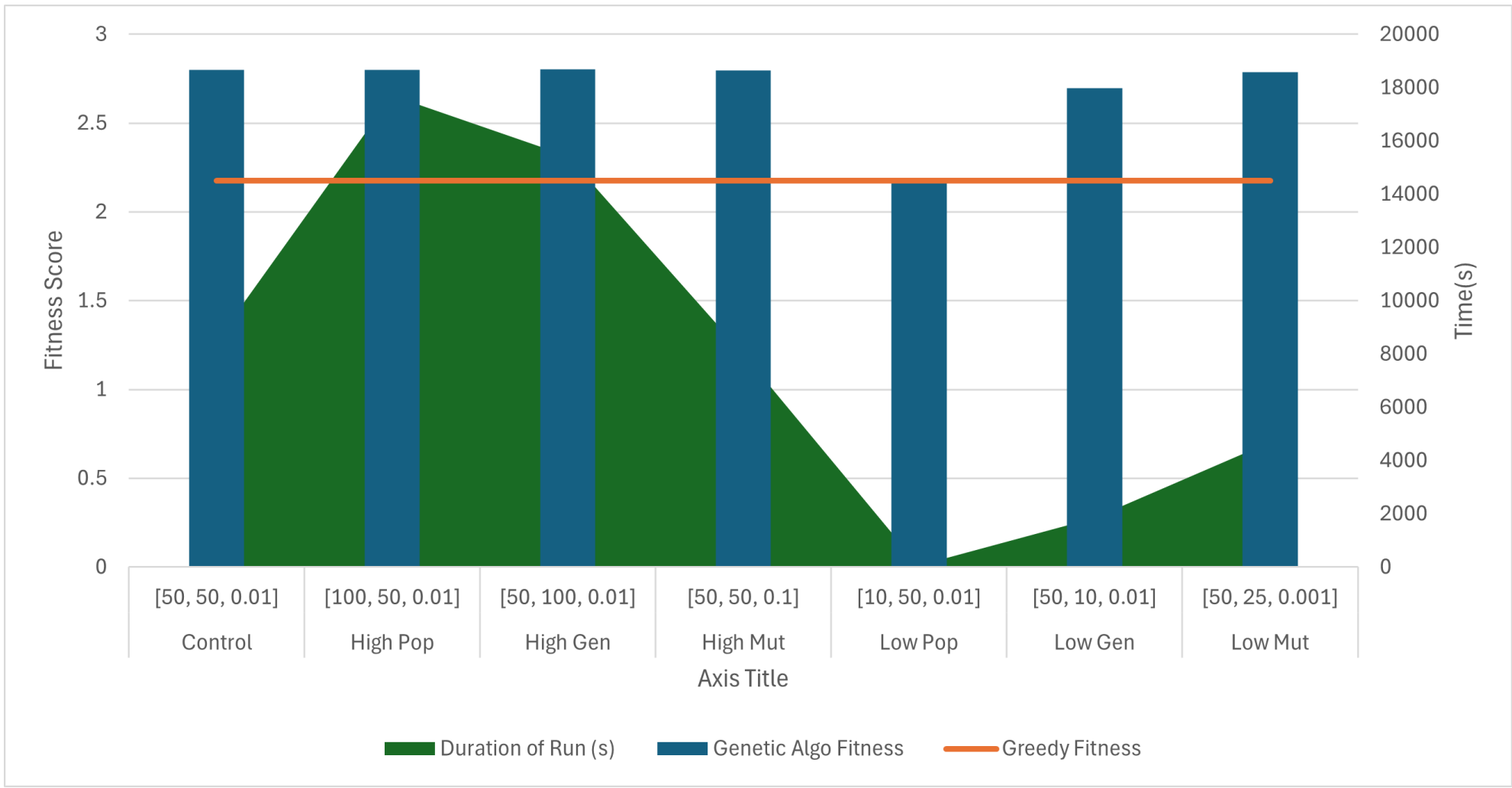
Figure 1: Genetic Algorithm performs 28.8% better than a greedy algorithm on the same data

Can the Genetic Algorithm be Improved?

- There are a number of parameters that can be changed to affect the effectiveness of a genetic algorithm:

1. Population Size
2. Generation Count
3. Mutation Rate
4. Academic Count
5. Ratio of Papers to Academics

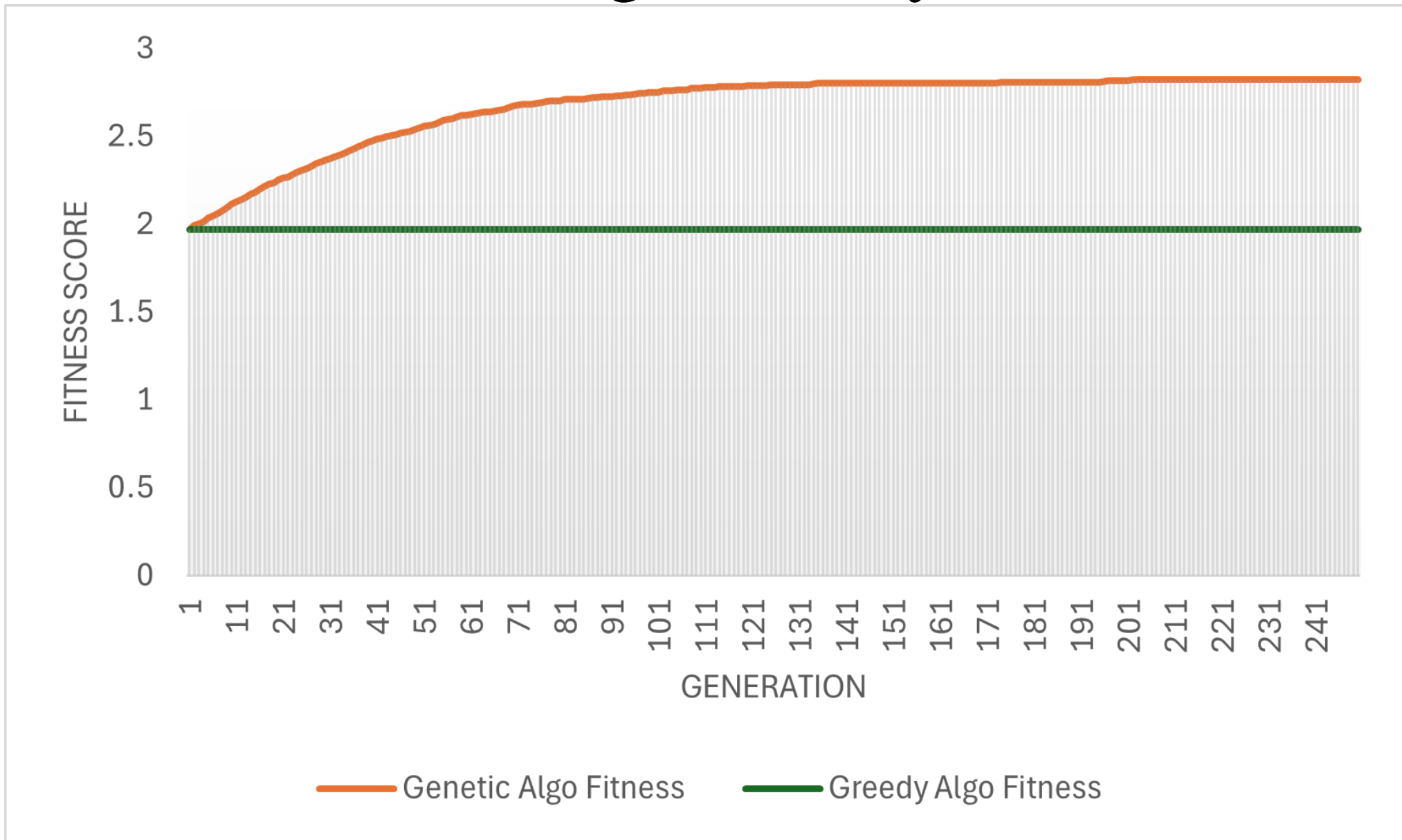
By altering these parameters individually, their effect can be seen more clearly:



Looking at these results, there are some takeaways for an effective genetic algorithm in this use-case:

- Maximise
 - generation count
 - ratio of papers to academics,
- Minimise
 - academic count
 - mutation rate
 - population size (while maintaining sufficient population to effectively explore potential solutions).

If all of this is brought together, the genetic algorithm functions significantly better:



Final values were:

- Final Fitness - 2.821457166
- % increase in fitness - 44.71979718%
- % increase in fitness per second - 0.126456669%/s
- Runtime - 353.6373186s

Previous best values were:

- Final Fitness - 2.8521 (runtime of 7569.5 seconds)
- % increase in fitness per second - 0.029348%/s (only a total % increase of 27.444%)
- Runtime - 53.408s (only total % increase of 0.25%)

Conclusion

In this Use-Case, genetic algorithms can significantly outperform greedy algorithms, when applied correctly (reaching up to 46.5% in ideal conditions). This represents an opportunity for a major increase in average score and therefore funding and an excellent example of where genetic algorithms thrive (multi-factor optimisation on large datasets).